Section 8.7: Cell-Based Architecture

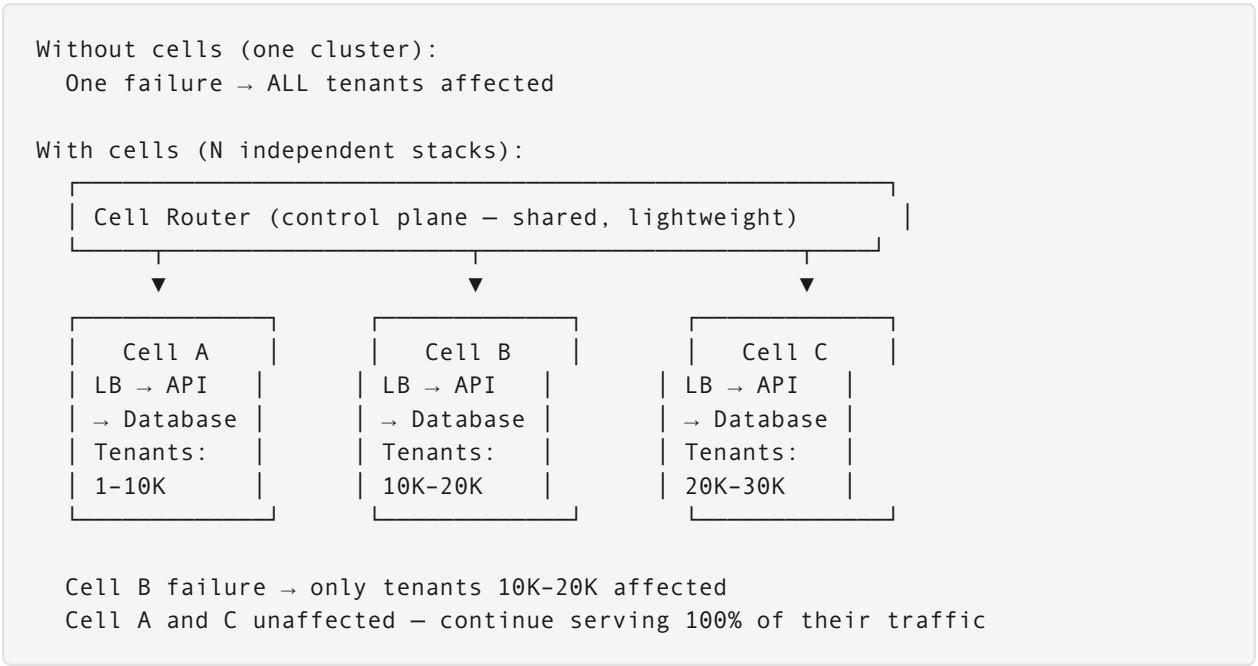
> ***Module 8 of 8 · Blast Radius Isolation, Shuffle Sharding & Failure Containment***

Executive Overview

Cell-based architecture limits the blast radius of failures by partitioning the system into independent, self-contained deployment units called cells. Each cell is a complete application stack: it has its own load balancer, compute tier, and data store. When one cell fails, only the tenants assigned to that cell are affected — all others continue without interruption. This section covers how cells work, how to route traffic to them, and how to handle outliers like celebrity accounts.

Part 1: Cellular Isolation

1.1 What Is a Cell?



1.2 What Goes Inside a Cell vs Shared

Component	Inside Each Cell	Shared (Control Plane)
Load balancer	✓	—
Application servers	✓	—
Database (primary + replica)	✓	—
Cache (Redis)	✓	—
Message queue	✓	—
Tenant routing table	—	✓
Billing service	—	✓
Analytics / reporting	—	✓
Provisioning / admin	—	✓

The control plane is kept minimal and highly available. All data-plane work happens inside cells.

1.3 Cell Sizing

```
Typical cell sizing considerations:

AWS: ~2 AZs per cell, 3-5 app server instances, 1 RDS primary + 1 replica
Tenants per cell: 10K-100K (depending on workload)
Total cells: 10-1,000 (depending on scale)

Blast radius: failure affects = (1 / num_cells) × 100% of tenants
Example: 100 cells → 1% of tenants affected per cell failure
```

Part 2: Shuffle Sharding

2.1 Problem With Simple Hashing

```
Simple hash: cell = hash(tenant_id) % N

Problem 1 — Hot tenant:
  Tenant X is 10× busier than average
  Tenant X always lands on Cell 5
  Cell 5 is overwhelmed; other cells idle
```

```
Problem 2 — Correlated failures:
  All tenants in Cell 5 are affected by one noisy tenant
  1/N of all tenants → bad experience simultaneously
```

2.2 Shuffle Sharding Solution

```
Shuffle sharding: each tenant is assigned a random SUBSET of cells
Each request goes to one cell from the tenant's subset

Tenant A assigned cells: {1, 5, 12, 34}
Tenant B assigned cells: {2, 7, 15, 40}

Probability Tenant A and Tenant B share a cell:
  P = C(N-subset, subset-1) / C(N, subset)

  N=100 cells, subset=4:
  P = C(96, 3) / C(100, 4) = (96×95×94)/(6) / (100×99×98×97)/(24)
  P ≈ 0.06%

  Only 0.06% chance a bad tenant's cell overlaps any given other tenant
  (vs 4% with simple hashing using 4/100 cells)
```

```
def assign_cells(tenant_id: str, num_cells: int, cells_per_tenant: int) ->
list[int]:
    """Assign a stable random subset of cells to a tenant."""
    rng = random.Random(tenant_id)  # Deterministic: same tenant → same cells
    return sorted(rng.sample(range(num_cells), cells_per_tenant))

def route_request(tenant_id: str, request_id: str) -> int:
    """Route a specific request to one of the tenant's assigned cells."""
    cells = assign_cells(tenant_id, num_cells=100, cells_per_tenant=4)
    # Consistent: same request_id → same cell (useful for retry)
    index = murmur_hash(request_id) % len(cells)
    return cells[index]

# Result: 4-cell shuffle sharding over 100 cells
# Single cell failure affects ≈ 0.06% of tenants (vs 4% with simple hash)
```

Part 3: Cell Routing & Failover

3.1 Routing Architecture

```
Client → DNS → Global Load Balancer → Cell Router → Cell
                                          |
                                reads: tenant_cell_map
```

```
                                (updated by control plane)

tenant_cell_map (in Redis / DynamoDB):
  tenant_123 → [cell_05, cell_12, cell_34, cell_78]  # shuffle shard
  tenant_456 → [cell_01, cell_23, cell_67, cell_89]
```

```
class CellRouter:
    def __init__(self, cell_map: CellMapping, health_checker: HealthChecker):
        self.cell_map = cell_map
        self.health = health_checker

    def route(self, tenant_id: str, request_id: str) -> CellEndpoint:
        candidate_cells = self.cell_map.get_cells(tenant_id)

        # Filter to healthy cells
        healthy_cells = [c for c in candidate_cells if self.health.is_healthy(c)]

        if not healthy_cells:
            # All assigned cells unhealthy — route to best available cell
            # (tenant may see stale data; acceptable vs total outage)
            return self.emergency_fallback(tenant_id)

        # Consistent hashing within healthy cells for request affinity
        cell_index = murmur_hash(request_id) % len(healthy_cells)
        return healthy_cells[cell_index]

    def emergency_fallback(self, tenant_id: str) -> CellEndpoint:
        # Find any healthy cell and temporarily reassign tenant
        available = self.health.get_all_healthy()
        if not available:
            raise NoHealthyCellsError()
        # Assign deterministically (same result on all router instances)
        return available[murmur_hash(tenant_id) % len(available)]
```

3.2 Cell Health Checking

```
class CellHealthChecker:
    def is_healthy(self, cell_id: str) -> bool:
        metrics = self.metric_store.get_cell_metrics(cell_id, window_seconds=60)

        return (
            metrics.error_rate < 0.01          # < 1% error rate
            and metrics.p99_latency_ms < 500   # p99 < 500ms
            and metrics.cpu_utilisation < 0.90 # < 90% CPU
            and metrics.is_reachable           # ping check
        )
```

Part 4: Interview Design Problems

Problem 1 — Social Media Platform with Celebrity Accounts

Question: Design a cell-based system for a social media platform with 100M users. How do you handle users who become extremely popular (celebrity accounts with 10M+ followers)?

Solution:

```
# Step 1: Basic cell assignment for normal users
def assign_user_cell(user_id: str, total_cells: int = 1000) -> int:
    return murmur_hash(user_id) % total_cells

# Step 2: Classify user hotness at write time
def get_user_tier(user_id: str) -> str:
    follower_count = follower_cache.get_count(user_id)
    if follower_count > 1_000_000:  return 'celebrity'    # 10M+ followers
    if follower_count > 100_000:    return 'influencer'   # 100K-1M
    return 'normal'

# Step 3: Fan-out strategy by tier
class FeedFanOutService:
    def publish_post(self, author_id: str, post: Post):
        tier = get_user_tier(author_id)

        if tier == 'normal':
            # Push fan-out: write post to each follower's feed immediately
            # ~500 followers × ~10 cells = ~5,000 writes (manageable)
            followers = follower_service.get_followers(author_id)
            for follower_id in followers:
                cell = assign_user_cell(follower_id)
                cell_store.append_to_feed(cell, follower_id, post)

        elif tier == 'influencer':
            # Hybrid: push to active followers, pull for inactive
            active_followers = follower_service.get_active_followers(author_id,
days=7)
            for follower_id in active_followers:
                cell = assign_user_cell(follower_id)
                cell_store.append_to_feed(cell, follower_id, post)
            # Inactive followers fetch on next login (pull)

        else:  # celebrity
            # Pull fan-out only: post stored in celebrity's cell
            # Followers fetch celebrity's posts when loading feed
            cell = assign_user_cell(author_id)
            cell_store.store_post(cell, author_id, post)
            # Async: replicate to read-replica cells for locality
            self.replicate_celebrity_post(author_id, post)

    def replicate_celebrity_post(self, celebrity_id: str, post: Post):
        # Replicate to N cells where followers are concentrated
```

```
        top_cells = follower_distribution_cache.get_top_cells(celebrity_id,
top_n=20)
        for cell_id in top_cells:
            cell_store.replicate_post(cell_id, celebrity_id, post)
```

Feed read for celebrity content:

```
def build_user_feed(user_id: str) -> list[Post]:
    user_cell = assign_user_cell(user_id)

    # Local posts (people user follows who are normal/influencer)
    local_feed = cell_store.get_feed(user_cell, user_id)

    # Celebrity posts: read from local cell replica (if available)
    # or from celebrity's primary cell
    celebrity_ids = follower_service.get_followed_celebrities(user_id)
    celebrity_posts = []
    for celebrity_id in celebrity_ids:
        # Check if this cell has a replica of celebrity's posts
        posts = cell_store.get_posts_local_or_remote(
            local_cell=user_cell,
            source_user=celebrity_id
        )
        celebrity_posts.extend(posts)

    return merge_and_rank(local_feed, celebrity_posts)
```

Failure isolation for celebrity posts:

```
Celebrity Adele has 50M followers across 100 cells
Her posts are replicated to top-20 cells (where most followers live)

Cell 5 fails:
  - ~1% of normal users affected (their feed unavailable)
  - Celebrity posts: served from 19 other replica cells
  - Users in Cell 5 who follow Adele: degraded to primary cell (extra latency)
  - Zero data loss; ~3s additional latency for celebrity content
```

Problem 2 — Cell-Based Architecture for a Payment System

Question: Design a cell-based payment processing system. A critical bug is discovered in the payment processing code. How does the cell architecture help you limit blast radius during the fix?

Solution:

```
Cell structure for payments:
  100 cells, each handling ~1% of transaction volume
```

```
Bug scenario:
  Payment code version 2.4.1 has a bug that causes double-charges
  Discovered at 10:00 AM after 2% of transactions in Cell 7 were affected

  Without cells: must immediately halt all payment processing globally
  With cells:
    10:00: Detect anomaly in Cell 7 error rates (+5% error rate)
    10:01: Cell 7 immediately isolated — traffic rerouted to other cells
           Only ~1% of volume affected
    10:02: Revert Cell 7 to version 2.4.0
    10:05: Cell 7 back online with old version; all traffic restored
    10:30: Fix deployed and validated in Cell 8 (canary)
    11:00: Fix rolled to all cells progressively (10 cells per 10 min)
```

```python
class CellDeploymentController:
    def progressive_rollout(self, new_version: str):
        """Deploy new version to cells progressively with automatic rollback."""
        cells = list(range(100))
        batches = [cells[i:i+10] for i in range(0, 100, 10)]  # 10 batches of 10

        for batch_idx, batch in enumerate(batches):
            self.deploy_to_cells(batch, new_version)
            time.sleep(300)  # Wait 5 minutes

            # Check health of deployed cells
            unhealthy = [c for c in batch if not self.health.is_healthy(c)]
            if len(unhealthy) > 2:  # More than 2/10 cells degraded
                # Rollback this batch and stop rollout
                self.rollback_cells(batch, previous_version)
                self.alert_on_call(f"Rollout halted at batch {batch_idx}")
                return

        print(f"Version {new_version} fully deployed across all 100 cells")
```

Problem 3 — Migrating From Monolith to Cell-Based Architecture

Question: You have a monolithic application serving 5M users. What's your step-by-step migration to cell-based architecture, and how do you do it without downtime?

Solution:

```
Phase 1: Add Cell Router (no data migration, no downtime)
  - Deploy cell router in front of existing monolith
  - All traffic → Cell Router → Monolith (single "cell")
  - Establish tenant_cell_map data structure
  - Duration: 1 sprint

Phase 2: Split into 5 cells (copy-on-write migration)
```

```
  - Provision 5 new database instances (copies of monolith DB)
  - Deploy 5 new application instances
  - Dual-write period: all writes go to monolith DB + target cell DB
  - Reads: still from monolith DB
  - Duration: 2-4 weeks

Phase 3: Read migration (tenant by tenant)
  - For tenant T: compare cell DB vs monolith DB (consistency check)
  - If consistent: flip tenant T's reads to target cell
  - Monitor error rate — rollback if anomalies
  - Repeat for all tenants over 4-6 weeks

Phase 4: Cut off monolith writes
  - Once all tenants reading from cells: stop dual-write
  - Monolith becomes read-only, then retired
  - Duration: 1 sprint

Phase 5: Scale to final cell count (10, 20, 100 cells)
  - Add cells as needed; reassign tenants via cell router update
  - No application changes; only cell_map updated
```

Quick Reference

Concept	Formula / Value
Blast radius (simple hash)	1 / num_cells of tenants
Blast radius (shuffle shard, k=4, N=100)	~0.06% overlap probability
Typical cells per tenant (shuffle shard)	2–4
Cell health check interval	10–30s
Cells in AWS production systems	10–1,000
Celebrity fan-out threshold	> 100K–1M followers
Progressive rollout batch size	10% of cells per wave